



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Día, Fecha:

12/02/2026

Hora de inicio:

17:20

Software Avanzado Seccion B

Kevin Raúl Pozuelos Estrada

Arquitectura de Software

Software Avanzado

Escuela de Ingenieria de Ciencias Y Sistemas
Facultad de Ingenieria
Universidad de San Carlos de Guatemala

Agenda



RECORDATORIOS



Dudas Iniciales



Anuncios Generales



¿Qué son lo microservicios?



Como diseñarlos



Buenas practicas



Dudas Finales



Espacio para dudas

CONTENIDO

¿Qué son los microservicios?)

¿Qué son los microservicios?

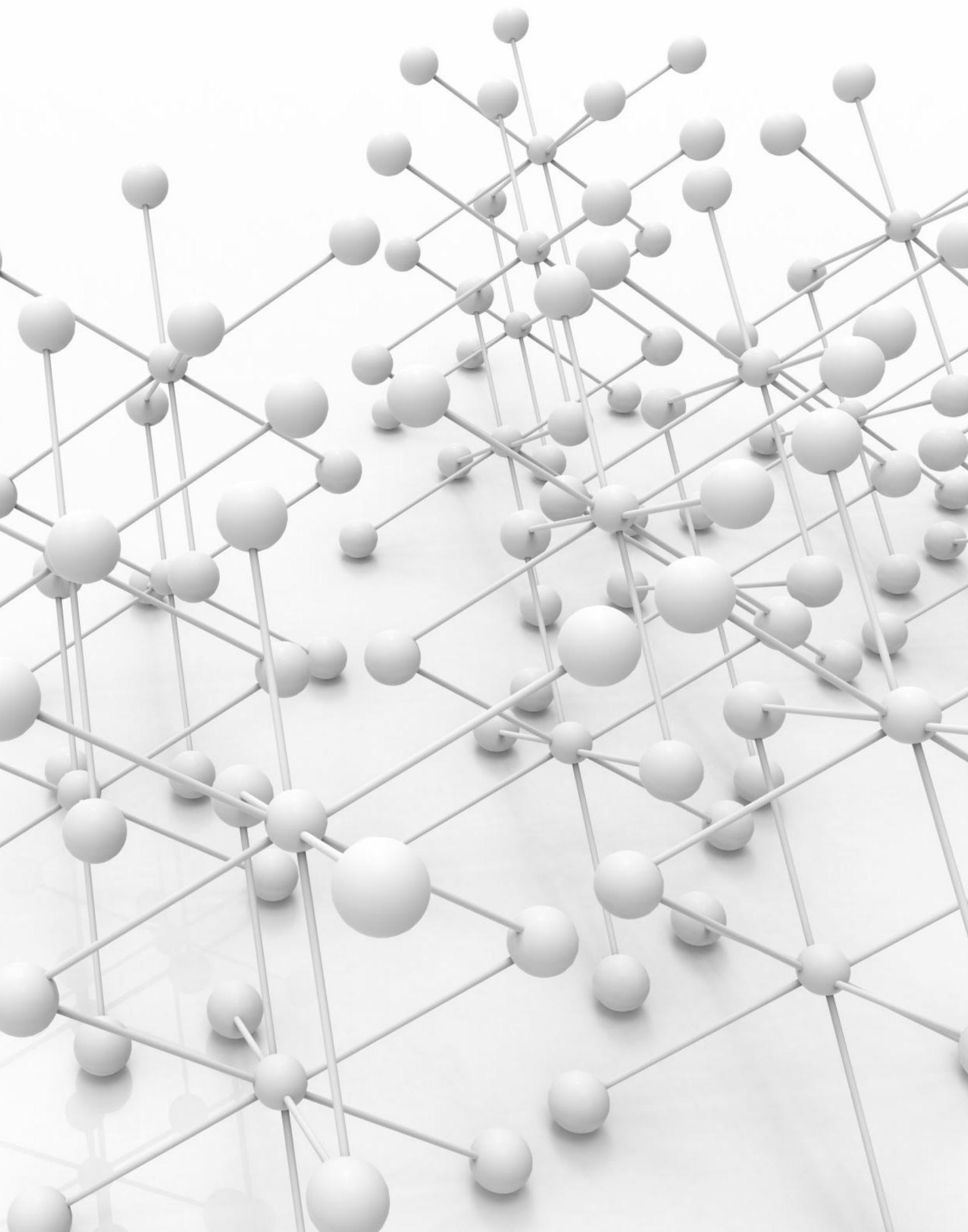
- Tipo de arquitectura de software donde una aplicación se divide en múltiples servicios pequeños e independientes. Cada uno de estos servicios se encarga de una función específica y puede desarrollarse, desplegarse y escalarse de manera autónoma.



Breve historia de los microservicios

- El término fue creado por un grupo de arquitectos de software en 2012, pero recién en 2014 empezó hacerse conocido, cuando Martín Fowler, desarrollador de software y autor conocido de la comunidad, comenzó a usar el término en algunas de sus publicaciones de su página web





Orígenes y primeros acercamientos

- **Arquitectura Monolítica (Años 90 - 2000s)**
 - Antes, la mayoría de las aplicaciones eran monolíticas, es decir, un solo bloque de código donde todos los componentes estaban integrados.
 - Problema: Dificultad para escalar y actualizar sin afectar toda la aplicación.
- **Arquitectura Orientada a Servicios (SOA) (2000s - 2010s)**
 - SOA promovió la idea de dividir las aplicaciones en servicios reutilizables que se comunican mediante protocolos como SOAP.
 - Problema: SOA era pesado y complejo, con grandes necesidades de infraestructura y gobernanza.

Factores que impulsaron los microservicios



Crecimiento del cómputo en la nube



DevOps y CI/CD

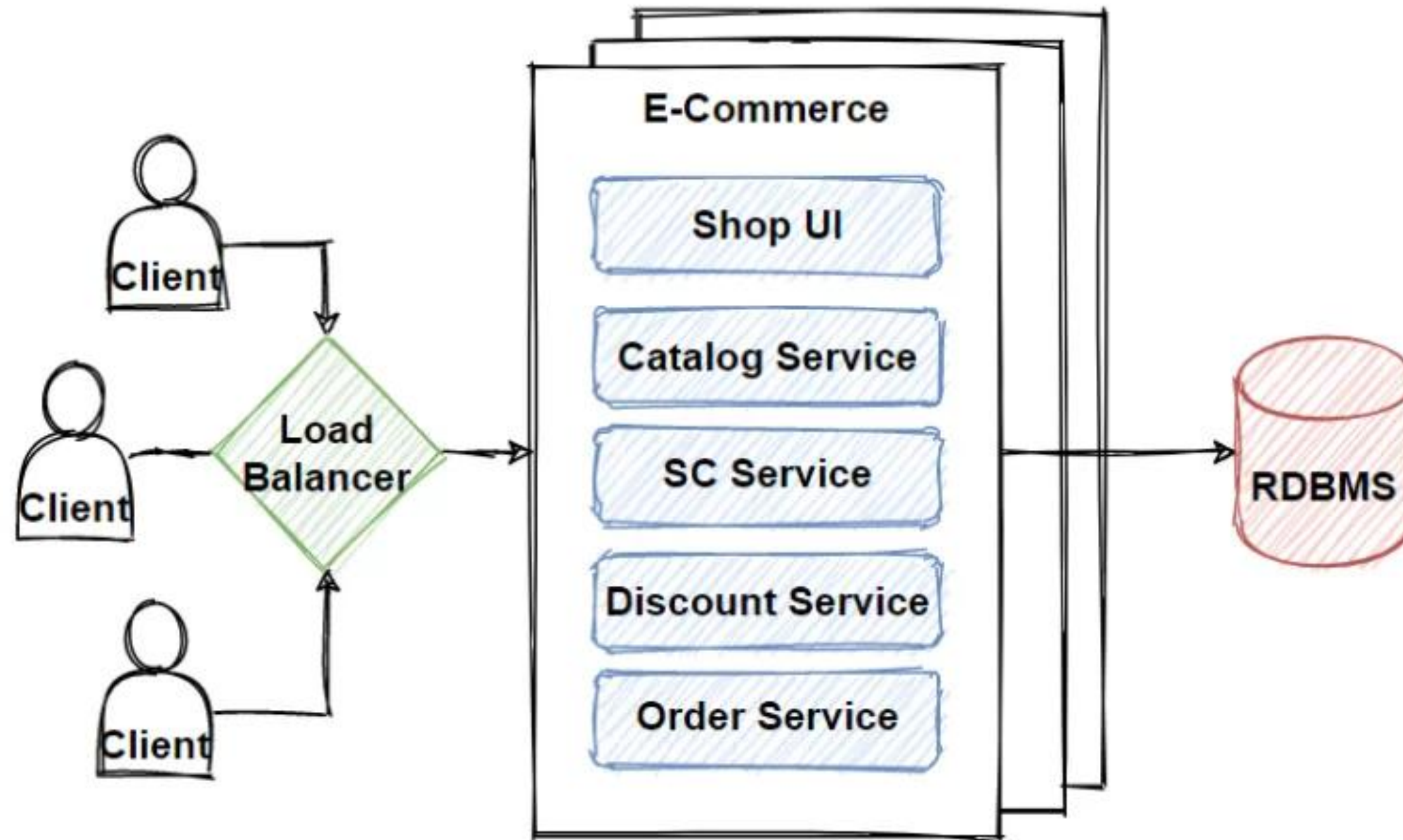


Contenedores y Kubernetes – Herramientas como Docker y Kubernetes



Necesidad de escalabilidad

Monolithic Architecture



Consumers Layer

Cloud Service
Consumers

Browser
(Human Users)

Enterprise Service Bus (ESB)

Providers Layer

Account
Service

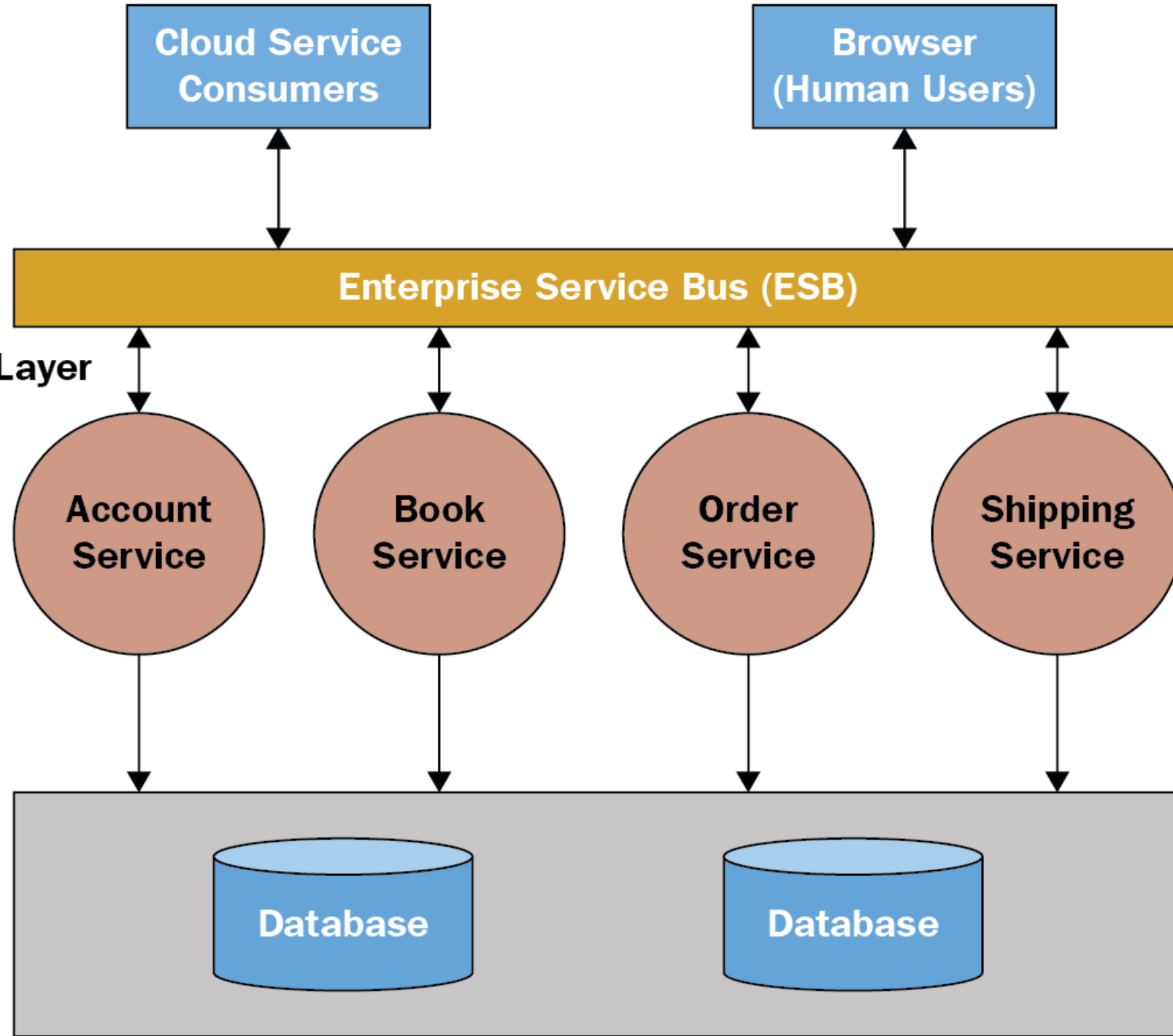
Book
Service

Order
Service

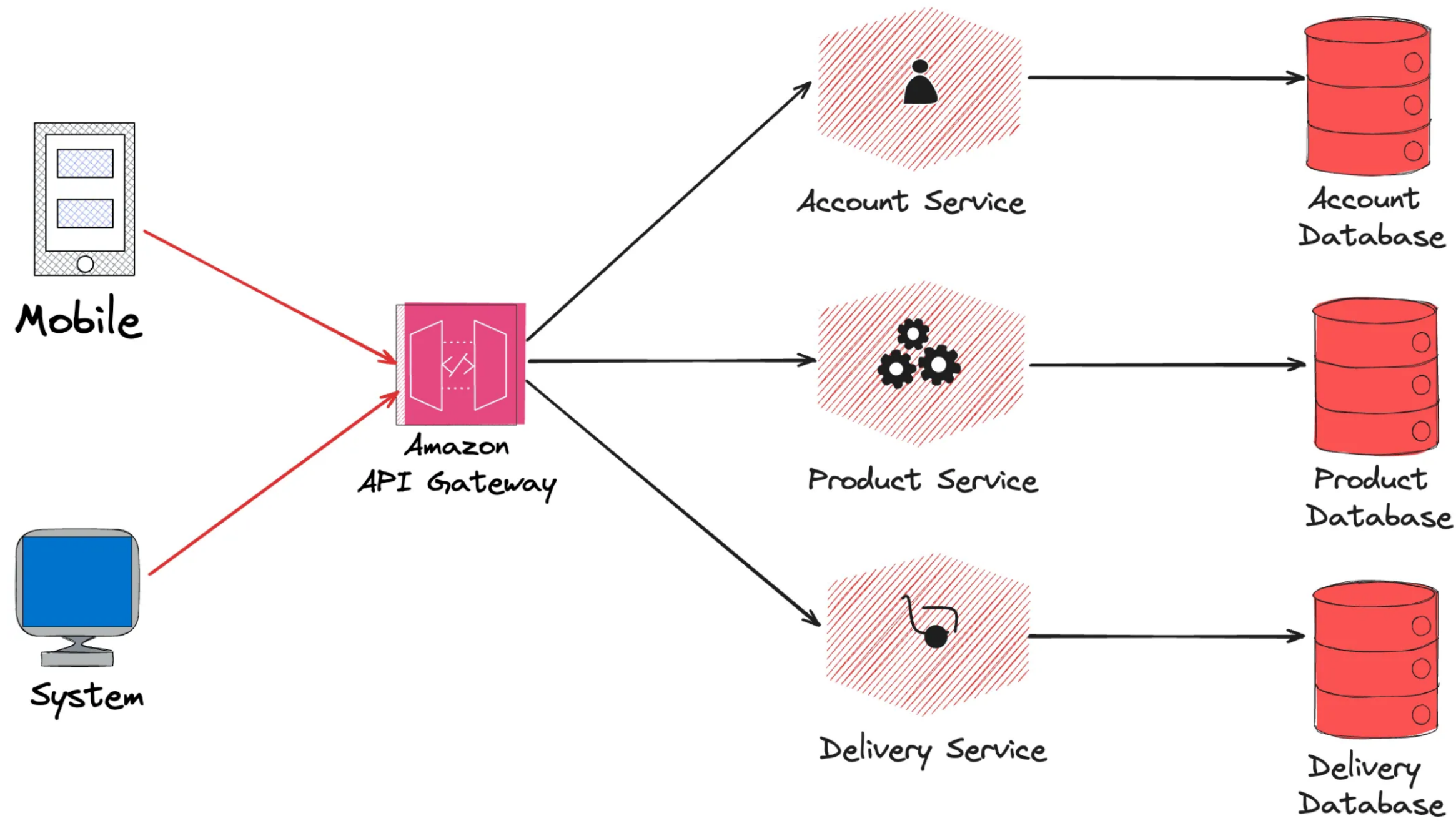
Shipping
Service

Database

Database



Microservicios



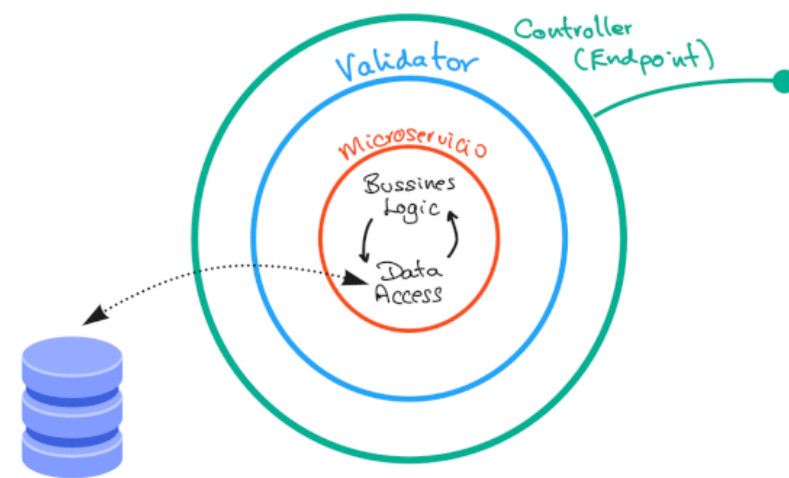
Ventajas

- Escalabilidad Independiente
- Desarrollo y Despliegue Ágil
- Flexibilidad Tecnológica
- Mayor Resiliencia y Tolerancia a Fallos
- Facilidad para Integraciones

Desventajas

- Mayor Complejidad en la Gestión
- Mayor Carga Operativa
- Dificultad en la Comunicación
- Problemas de Consistencia de Datos
- Tiempo de Desarrollo Inicial Mayor

Así se ve un microservicio



Requisitos de un microservicio

Cumplir una sola función



Ser autónomo



Estar aislado

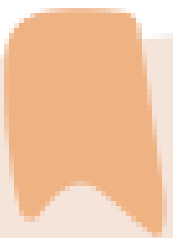


NOMBRE DEL VALOR

Recuerda que en una educación por competencias enseñamos valores indica ejemplos prácticos desde lo aprendido como van los valores aplicados

Más práctica, menos teoría
Hablar de valores puede ser aburrido si solo se queda en definiciones. Mejor usa casos reales, dinámicas, debates, videos, juegos de rol o storytelling.

Solamente coloca un valor y como se aplica este valor.



Nuestra casa de estudios cuenta con valores como Excelencia, Compromiso Responsabilidad, Ética Integridad, trata de relacionarlos a los temas que estas impartiendo

Cumplir una única función

Cada microservicio debe enfocarse en una única función o capacidad de negocio.

No deben existir servicios que hagan muchas cosas a la vez (evitar servicios gigantes o "macroservicios").

Ejemplo: Un servicio de autenticación solo debería encargarse del login y la seguridad, sin manejar pedidos o pagos.

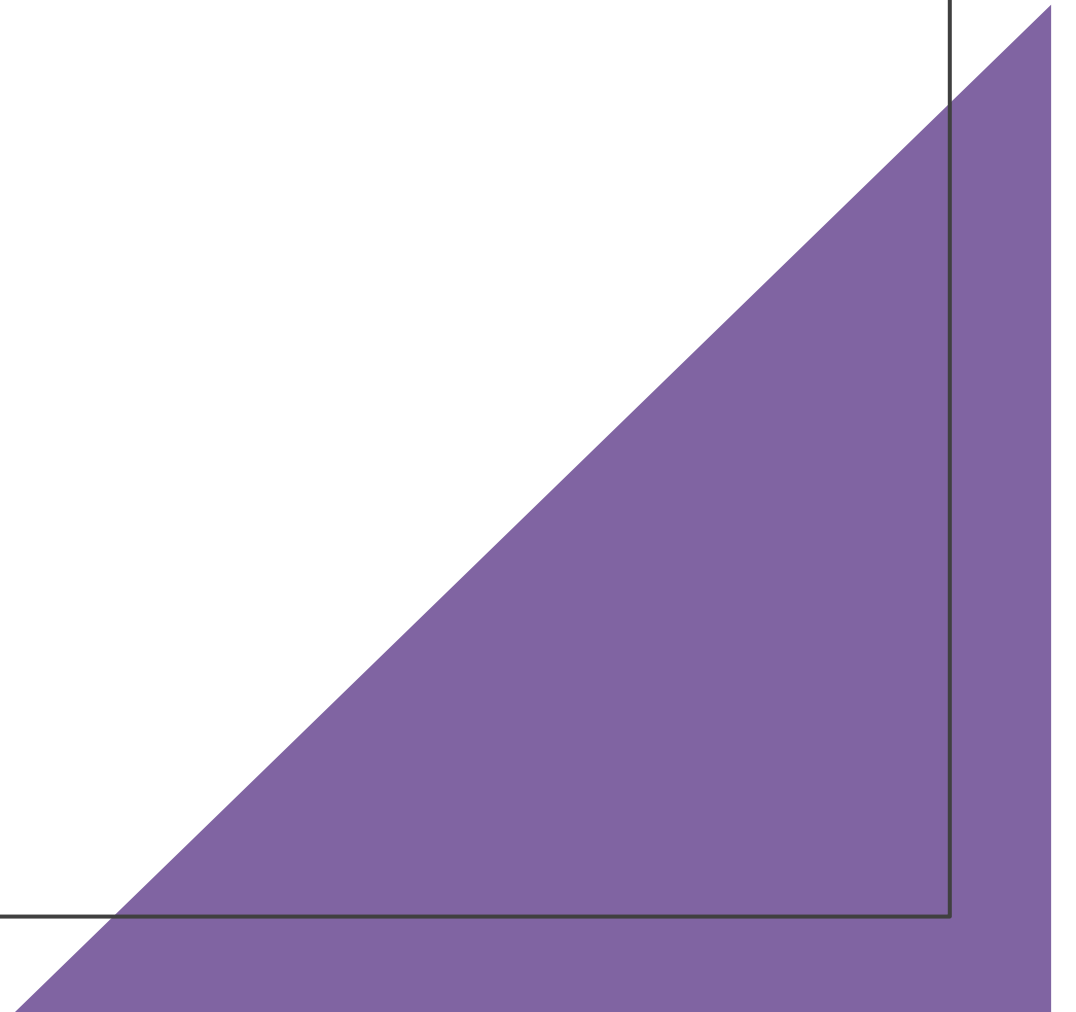
Ser Autónomo

- Cada microservicio debe ser independiente y autónomo, sin depender directamente de otros servicios para funcionar.
- Debe poder desarrollarse, desplegarse y escalarse sin afectar el resto del sistema.

Estar aislados

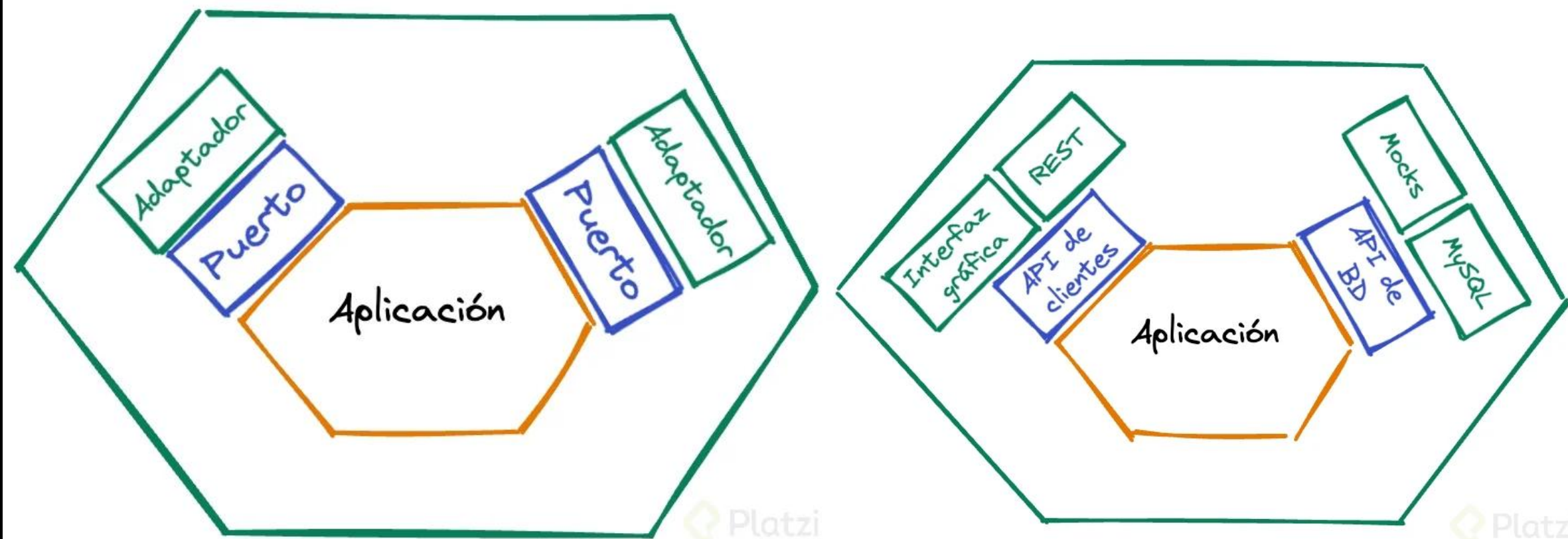
- Deben correr en entornos separados, con sus propios recursos y, muy recomendable, cada uno con su propia base de datos.

Herramientas y estrategias para diseño de microservicios



Arquitectura Hexagonal

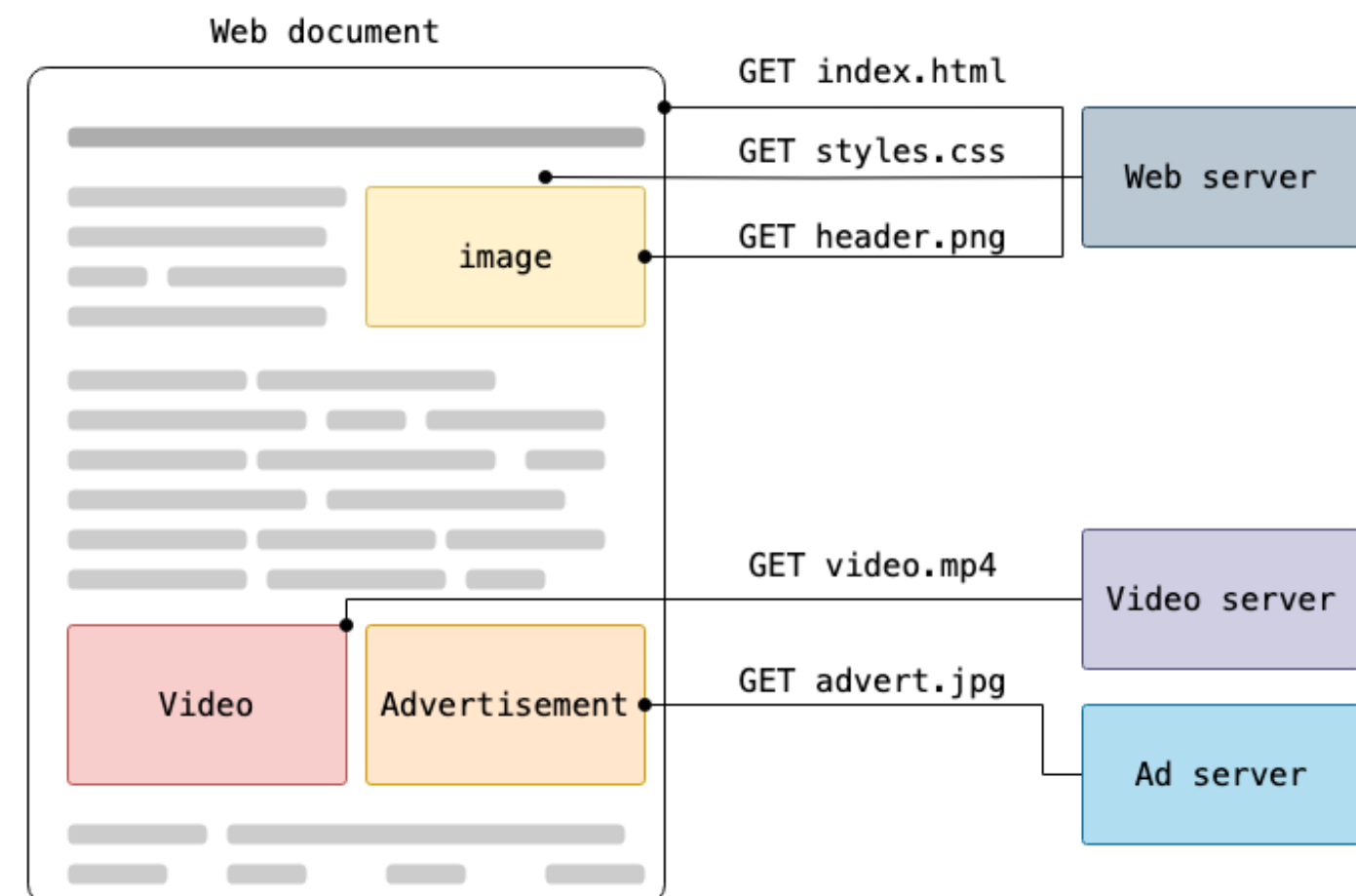
arquitectura basada en puertos y adaptadores para comunicar a los microservicios. Los puertos son interfaces que definen cómo se conectarán los microservicios y los adaptadores son implementaciones concretas de estos puertos.



Utilización de protocolos de comunicacion (HTTP, gRPC, eventos)

HTTP (Hypertext Transfer Protocol)

Protocolo que constituye los cimientos de la red mundial, y se utiliza para cargar páginas web mediante enlaces de hipertexto. HTTP es un protocolo de capa de aplicación diseñado para transferir información entre los dispositivos conectados de la red, y se ejecuta sobre otras capas del conjunto de protocolos de la red. Un flujo típico sobre HTTP implica una máquina cliente que realiza una solicitud a un servidor, que a continuación envía un mensaje de respuesta.

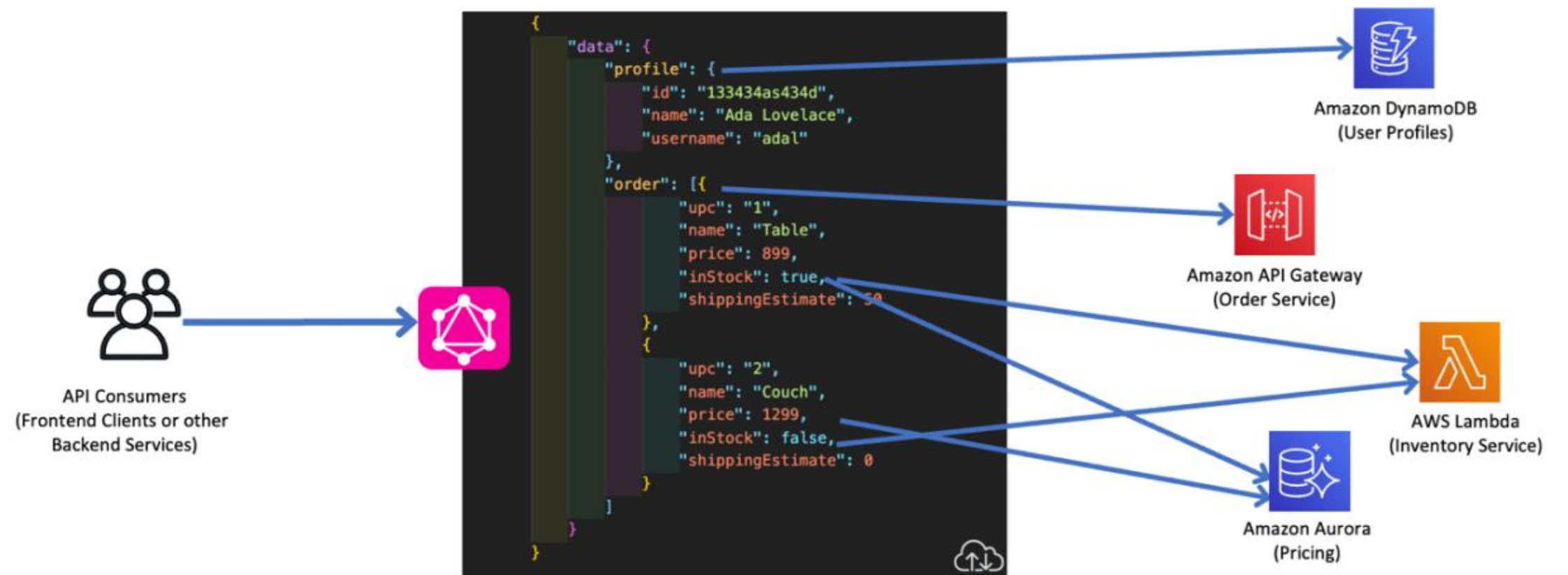


GraphQL como parte de HTTP y reemplazo de REST

GraphQL

Es un lenguaje de consulta y un tiempo de ejecución del servidor para las interfaces de programación de aplicaciones (API); su función es brindar a los clientes exactamente los datos que solicitan y nada más.

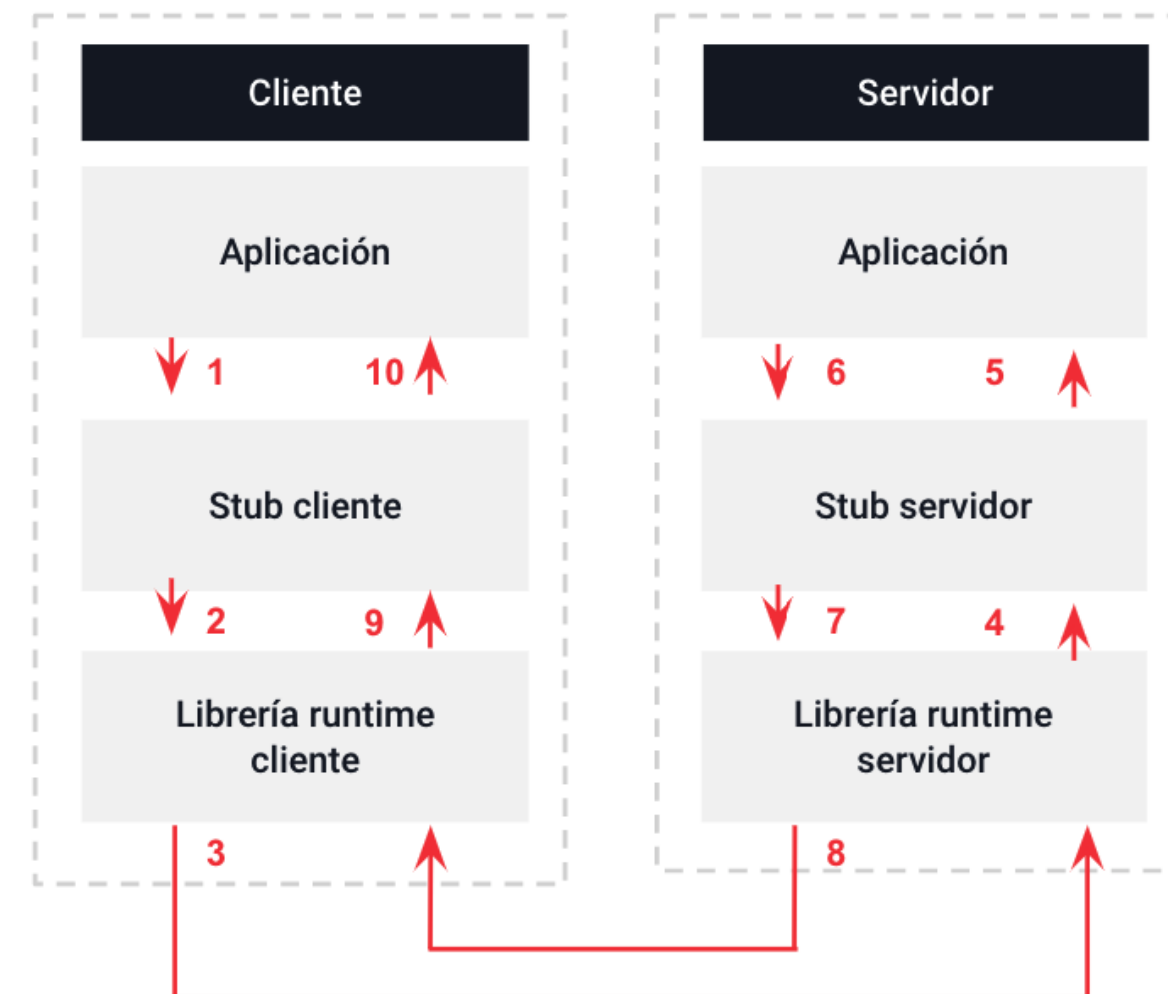
Como alternativa a REST, GraphQL permite que los desarrolladores creen consultas para extraer datos de varias fuentes en una sola llamada a la API.



Utilizacion de protocolos de comunicacion (HTTP, gRPC, eventos)

gRPC (Google Remote Procedure Call)

Se emplea en comunicaciones cliente-servidor distribuidas por su eficiencia gracias a la ingeniería de procesos basada en RPC. Este mecanismo permite la comunicación de una forma tan sencilla como si se tratara de una comunicación local entre procesos de una misma máquina.



Utilizacion de protocolos de comunicacion (HTTP, gRPC, eventos)

Eventos (Modelo de comunicación asincrónica)

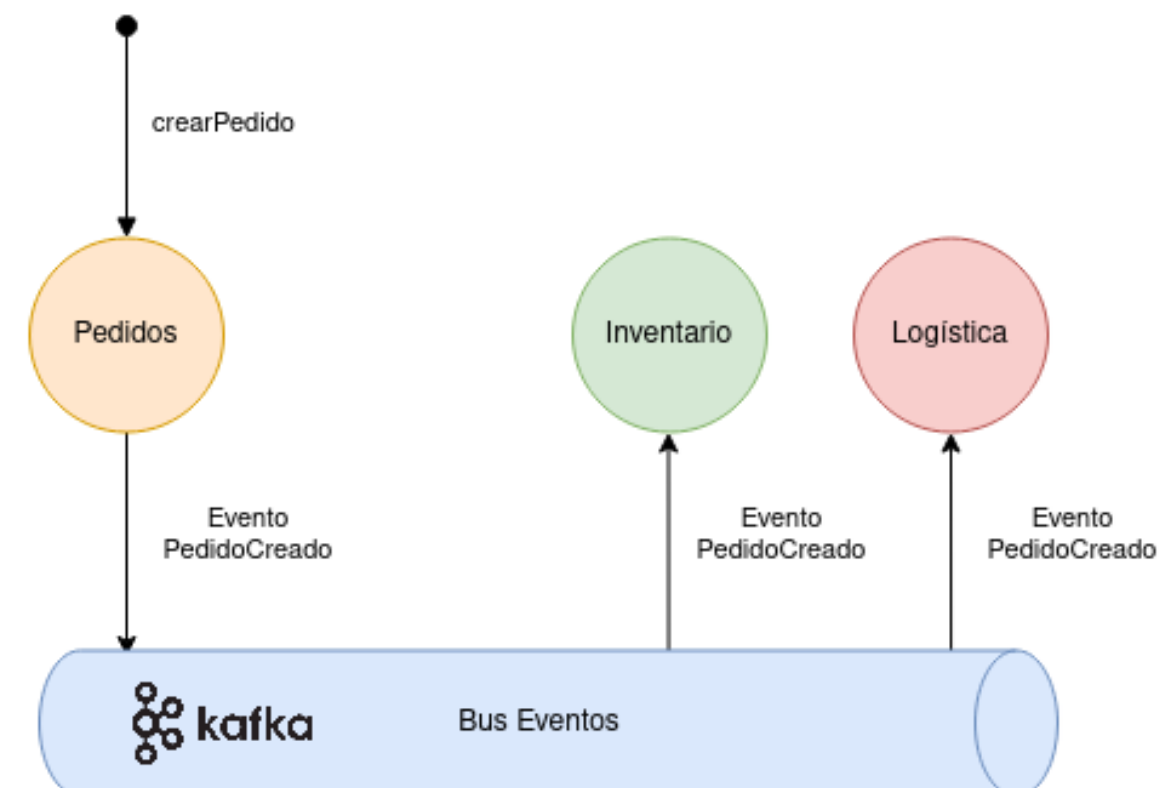
En lugar de peticiones y respuestas, los microservicios trabajarán con el concepto de eventos. Un evento es un mensaje o registro que representa un acontecimiento en el dominio de nuestra aplicación.

Al trabajar con eventos tendremos dos roles implicados, productores y suscriptores.

Productores: generan eventos en el sistema.

Suscriptores: consumen eventos y reaccionan a ellos.

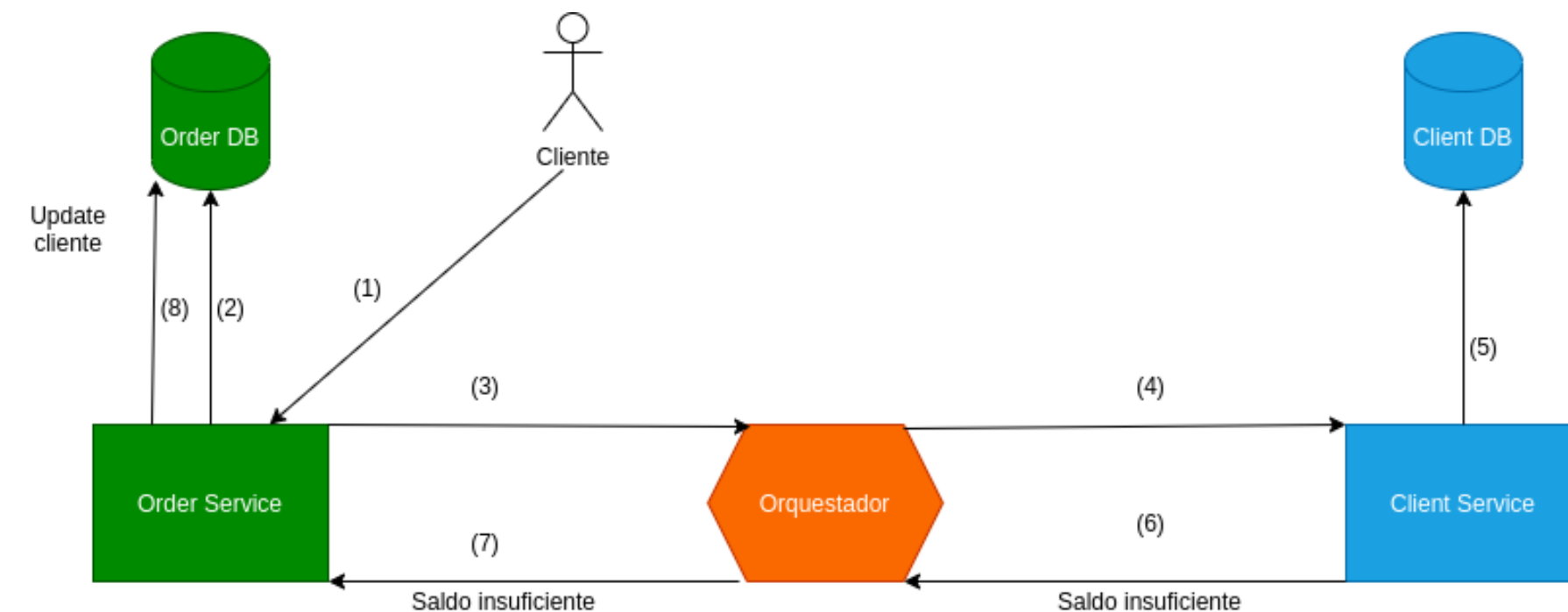
Un microservicio puede ser productor de eventos, suscriptor o ambos roles simultáneamente.



¿Qué es el Patrón Saga?

El patrón Saga es una estrategia de diseño que se utiliza para mantener la consistencia de las transacciones en sistemas distribuidos. En lugar de depender de una única transacción global que involucra múltiples servicios, el patrón Saga divide la transacción en una serie de pasos más pequeños y autónomos. Cada uno de estos pasos es responsabilidad de un servicio específico y se ejecuta dentro de su propia transacción. Si un paso falla, se desencadena un proceso de compensación para revertir los cambios realizados en los pasos anteriores.

Patrones de diseño: SAGA y Circuit Breaker

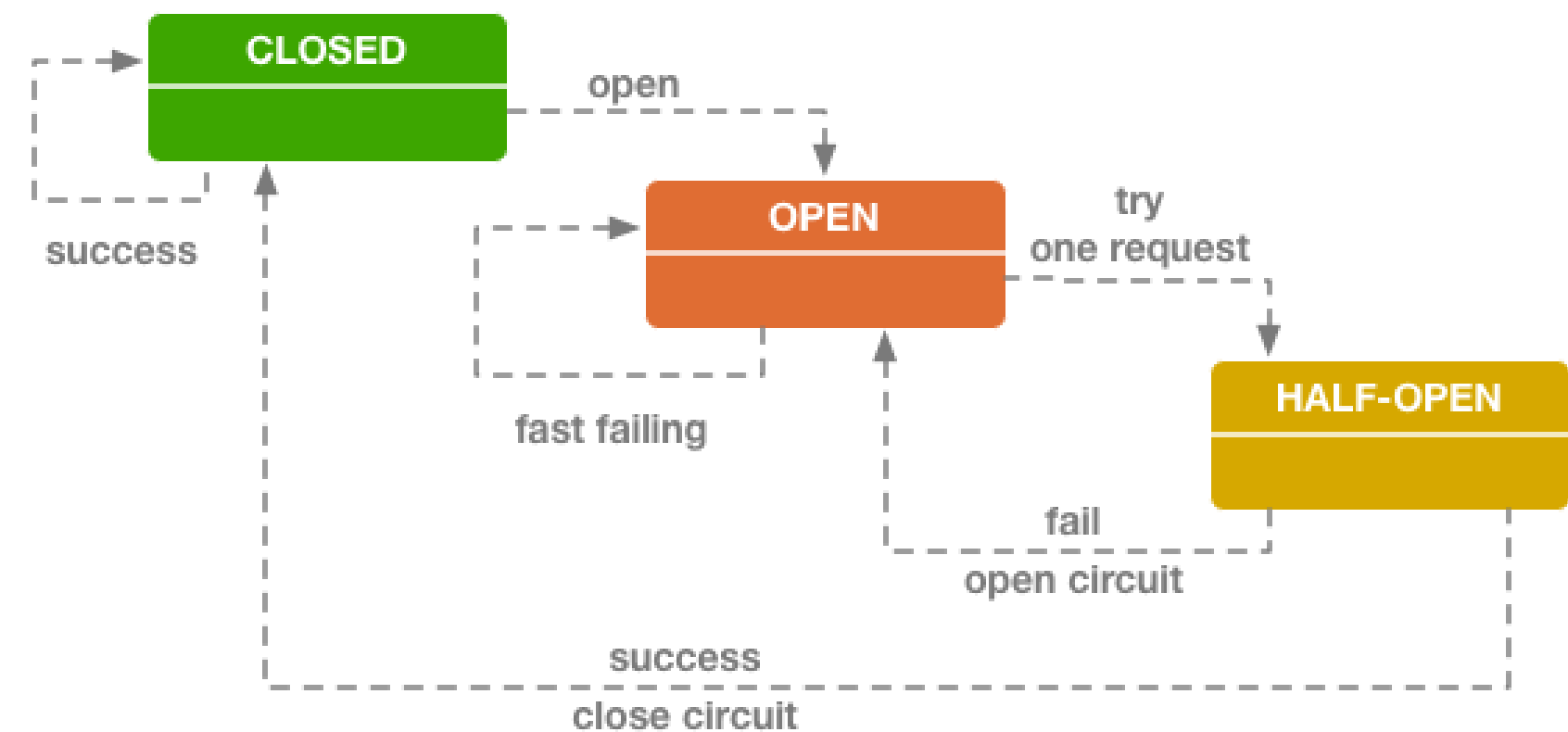


¿Qué es el Circuit Breaker?

Se define como una opción utilizada para detectar fallos y encapsula la lógica de prevenir que un fallo se repita constantemente durante el mantenimiento, un fallo temporal de sistema externo o dificultades inesperadas del sistema. Esto quiere decir que impide que un sistema lleve a cabo o continúe haciendo solicitudes a otro cuando existen altas posibilidades de que falle el proceso.

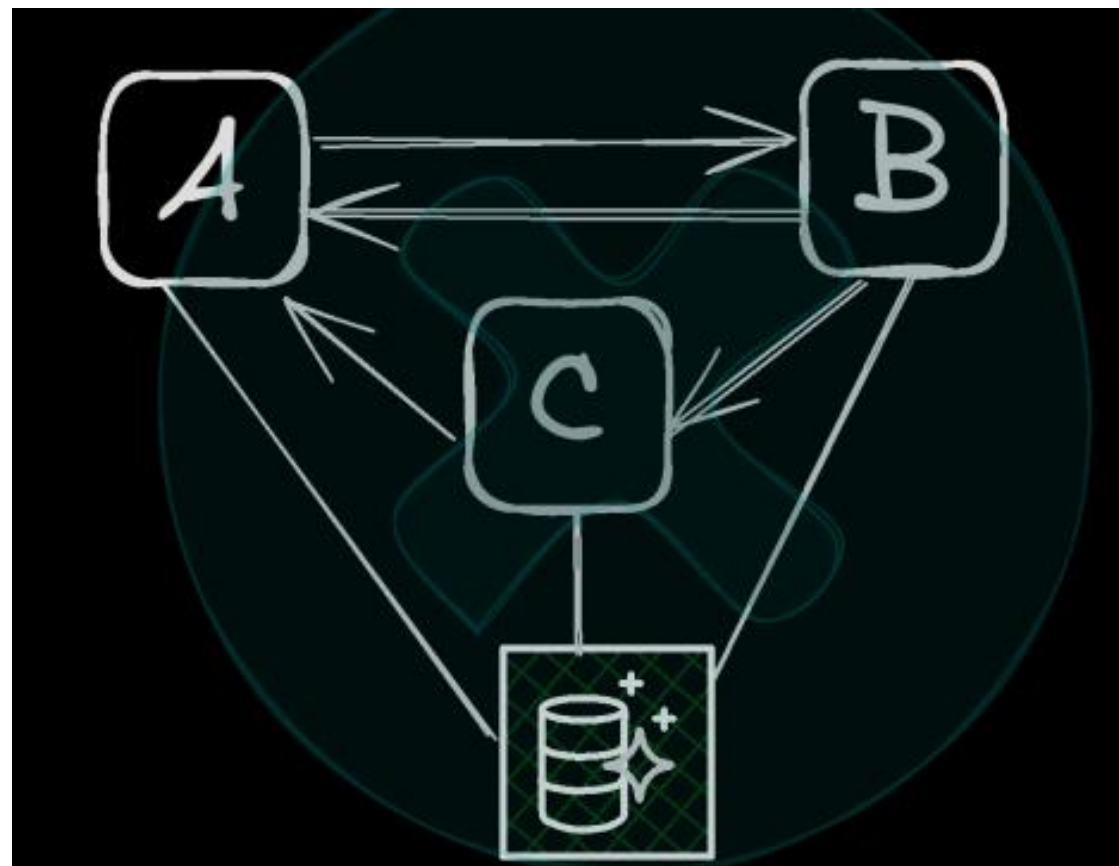
Además, permite el cierre de la comunicación con un servicio específico cuando se identifica que este se encuentra fallando, lo que implica que debe actuar como un proxy para las operaciones que pueden llegar a presentar errores.

Patrones de diseño: SAGA y Circuit Breaker



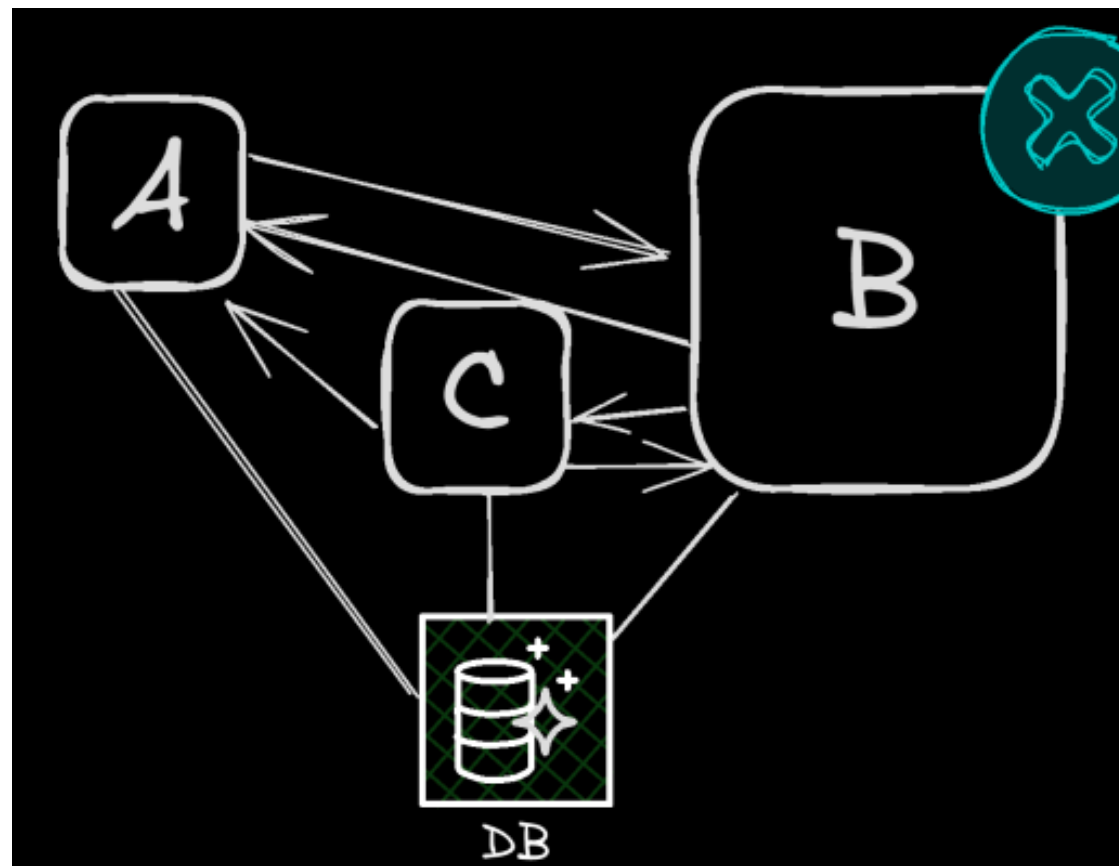
Circuit Breaker State Diagram

Antipatrones en los microservicios



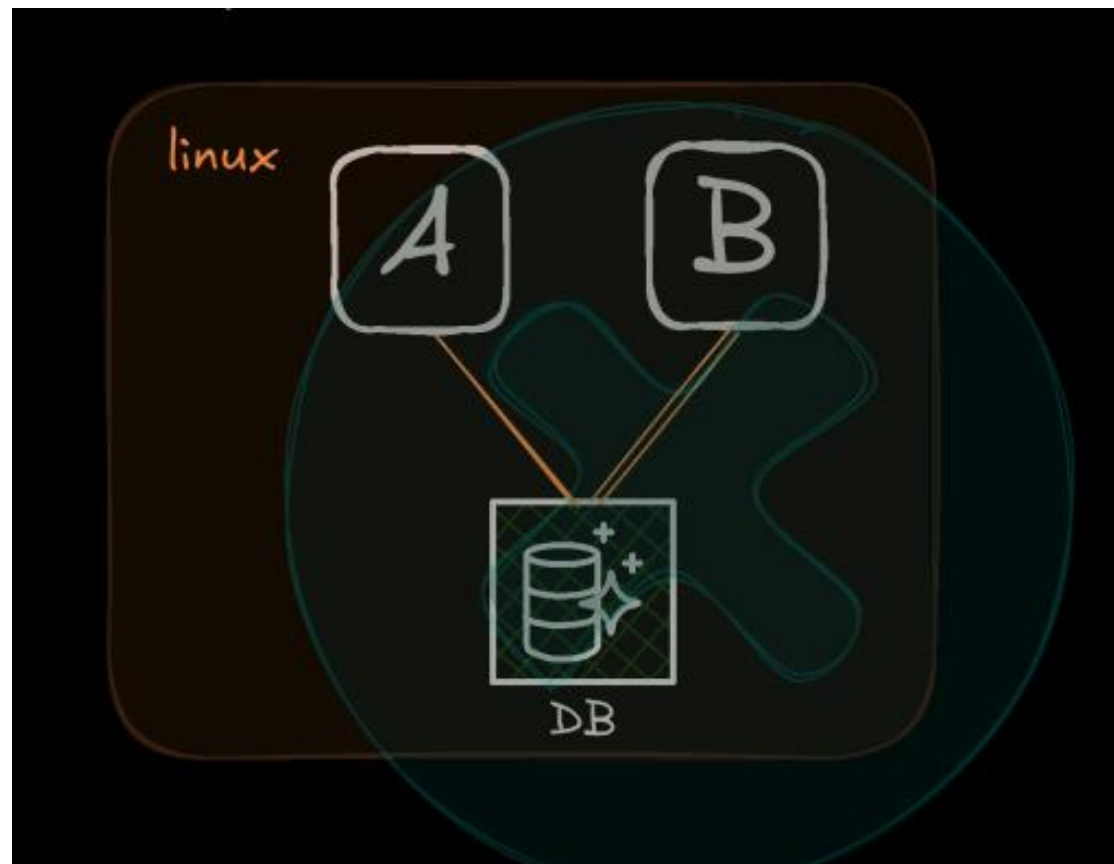
- Monolito Distribuido
- Evitar separar solo el código; separar la persistencia, aplicar una base de datos por servicio y usar la comunicación de forma adecuada

Antipatrones en los microservicios

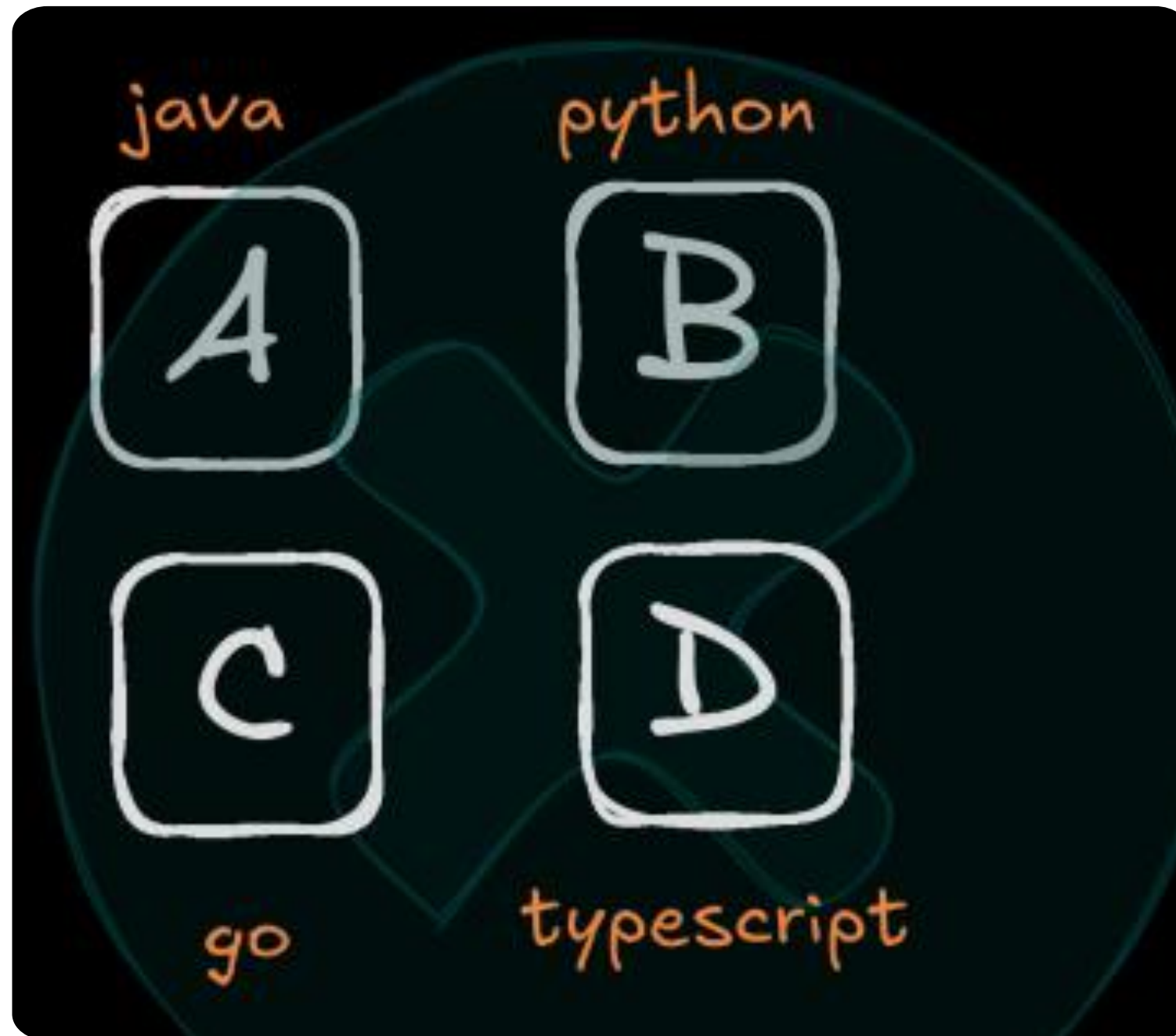


- Enfoque “el microservicio”
- No implementar un servicio “dios” que contenga toda la logica de negocio importante. Un servicio excesivamente grande es más difícil de manter y escalar

Antipatrones en los microservicios



- Infraestructura compartida
- Evitar que dos servicios usen la misma base de datos, caché o desplegados en el mismo nodo. Compartir recursos es riesgoso, ya que será más propenso a la falta de disponibilidad y más dificultad al escalar.

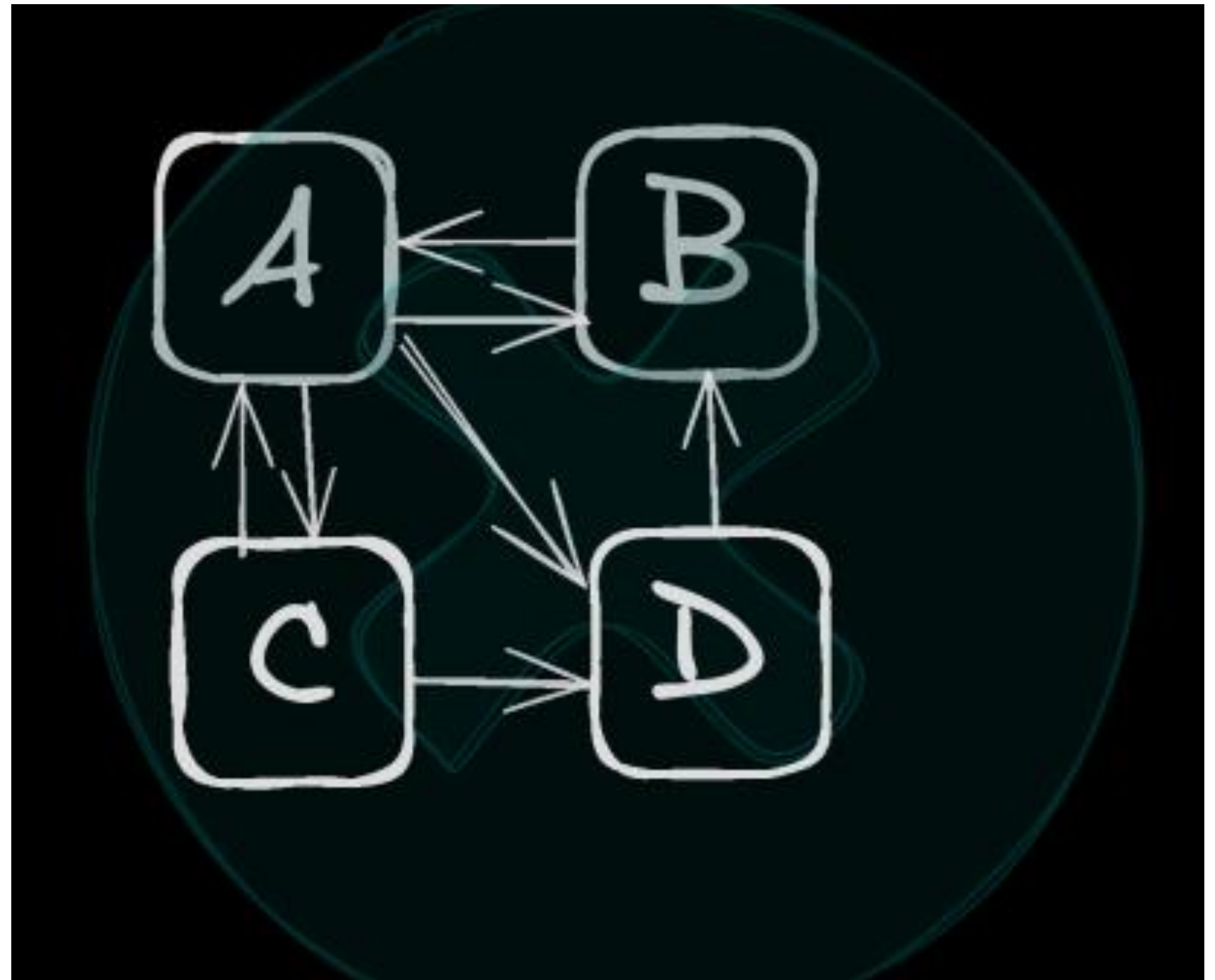


Antipatrones en los microservicios

- Excesivo uso de tecnologías
- Aunque sea una de las ventajas, no porque se pueda significar que debemos implementar una gran diversidad de tecnologías, ya que agregaría una capa de dificultad al mantenimiento además de la difícil tarea de conseguir un equipo sólido y capaz de resolver problemas que se presenten.

Antipatrones en los microservicios

- Comunicación Excesiva
- Reducir tanto como sea posible las llamadas síncronas entre servicios. Evitar dependencias cíclicas. Si es posible, usar la estrategia de mensajes/eventos.
- Dicho de otra forma, evitar al máximo el chatting.





DUDAS ¿?



**¡GRACIAS
ATENCIÓN!**

POR

SU

RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES CONSULTAR CUALQUIER DUDA
QUE TE SURJA EN LA SEMANA